Cloud Telephony Platform

Overview

This phase focuses on building a robust, scalable server infrastructure that handles user management, integrates with SYSCONFiG's APIs, processes payments, and provides REST APIs for all client applications. The backend serves as the central nervous system of our entire platform, coordinating between our user-facing applications and SYSCONFiG's telephony infrastructure. Everything our users do: from signing up to making calls to viewing their bill is controlled through this backend system.

The REST APIs We're Building

The REST API we build during this phase exposes all functionality that client applications will need. Authentication endpoints handle **user registration, login, logout, and token refresh**. User management endpoints provide access to **profile information, SIP credentials, and account settings. Call-related endpoints return call histor**y with filtering and pagination, individual call details, and recording access. Contact management endpoints support creating, reading, updating, and deleting user contacts. **Phone number endpoints enable searching available numbers**, purchasing numbers, and managing owned numbers. Voicemail endpoints list voicemails, retrieve audio files, and mark messages as read. Billing endpoints provide balance information, transaction history, and payment method management.

These APIs represent what our applications will consume. But to make them work, our backend needs to integrate deeply with SYSCONFiG's systems. Let's break down each integration requirement and why it matters.

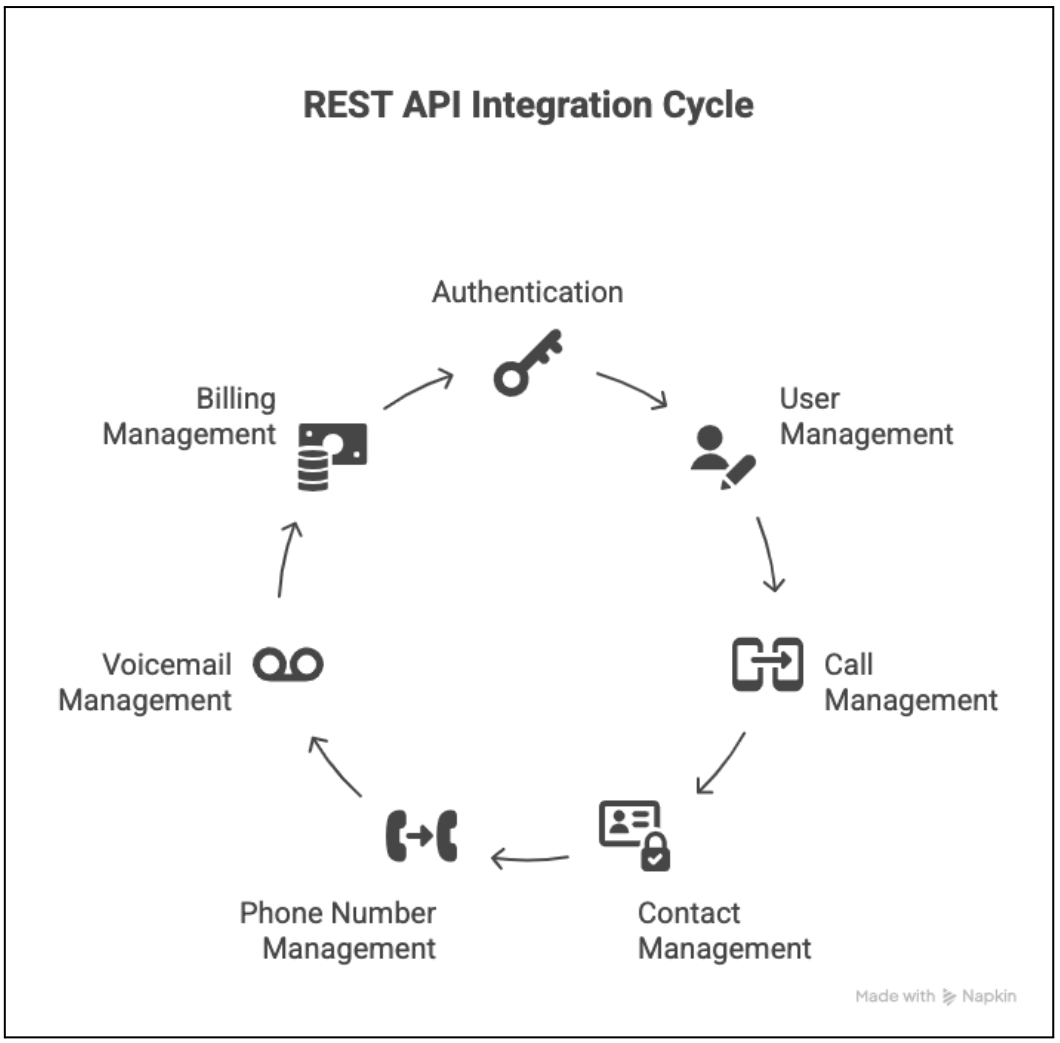

Fig. Rest API Integration Cycle

Real-Time Call Tracking Through Webhooks

Webhook handling represents another critical component. Every time a call ends, SYSCONFiG sends a webhook to our endpoint containing call detail records. These records include information about who called whom, call duration, cost, timestamps, and URLs to call recordings if applicable. Our webhook handler must authenticate incoming requests to ensure they genuinely come from SYSCONFiG, parse the payload data, validate the information, store it in our call_logs database table, and trigger any necessary downstream processes such as billing updates or user notifications. This happens in real-time for every call on the platform, so the webhook handler must be performant and reliable.

To set up this webhook system properly, SYSCONFiG needs to know where to send the data. The webhook configuration we need includes the URL where SYSCONFiG will post CDR data to our server, authentication method such as shared secret, HMAC signature, or API key to verify webhooks genuinely come from SYSCONFiG, retry policy explaining what

happens if our webhook endpoint is temporarily down, and delivery guarantee such as at-least-once delivery meaning we might receive duplicate webhooks.

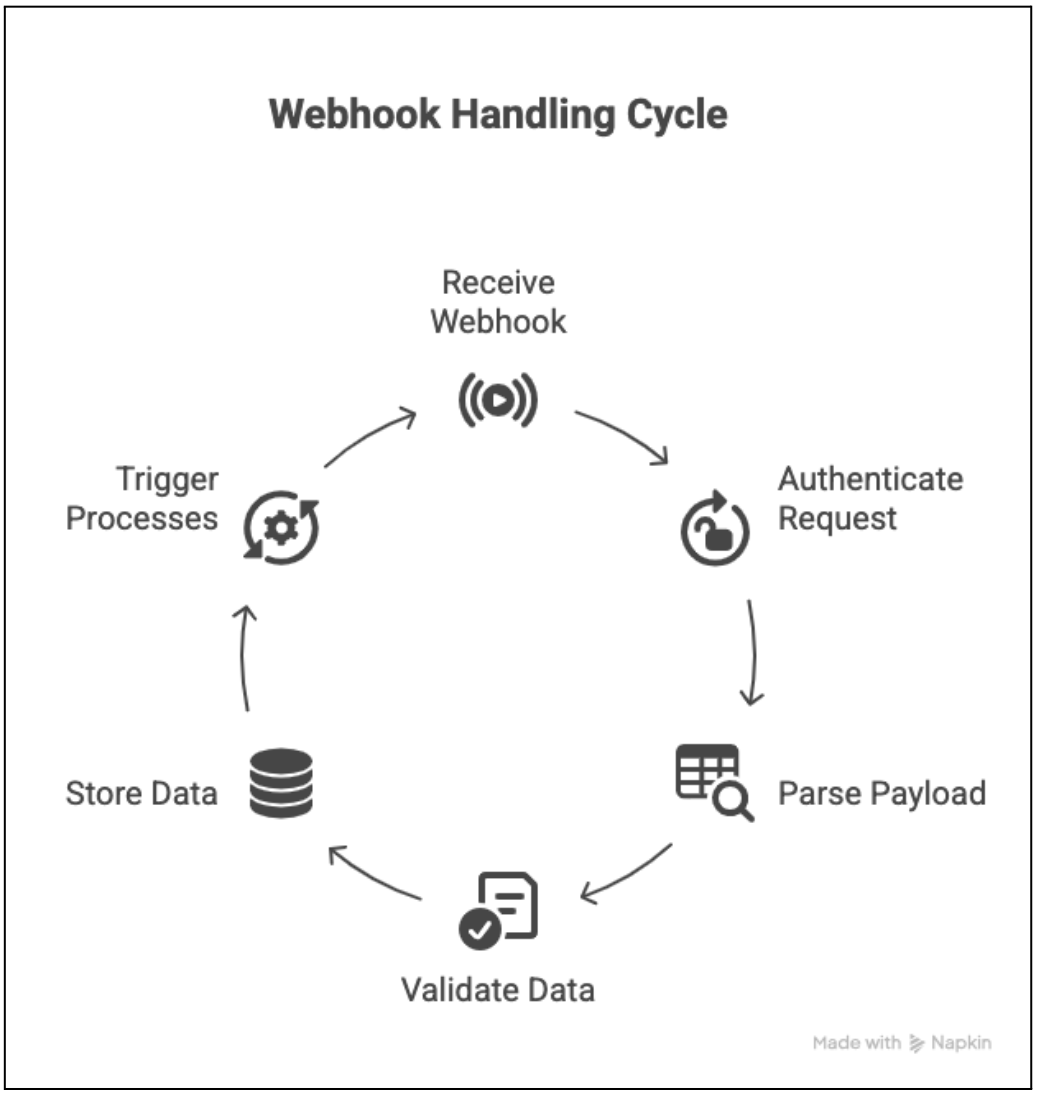

Fig. Webhook Handling Cycle

User Provisioning API

When we build our user registration system, the moment someone signs up on our platform, we must immediately create a SIP extension for them on SYSCONFiG's system. We need their user provisioning API that allows us to **create a new SIP extension** by sending user information like username and email, receive back SIP credentials including username, password, and SIP domain, update extension settings when users change their preferences, and delete extensions when users close their accounts. This API integration happens during user registration, so we need it in the very first week of backend development.

Specific API Endpoints Required:

- POST endpoint to create new SIP extension with parameters for username, email, and optional preferences
- GET endpoint to retrieve existing extension details using extension ID or username
- PUT or PATCH endpoint to update extension configuration like password reset or call forwarding settings
- DELETE endpoint to remove extensions for account closures

Once users have their accounts and SIP extensions set up, the next thing they'll want is their own phone number. This brings us to phone number management.

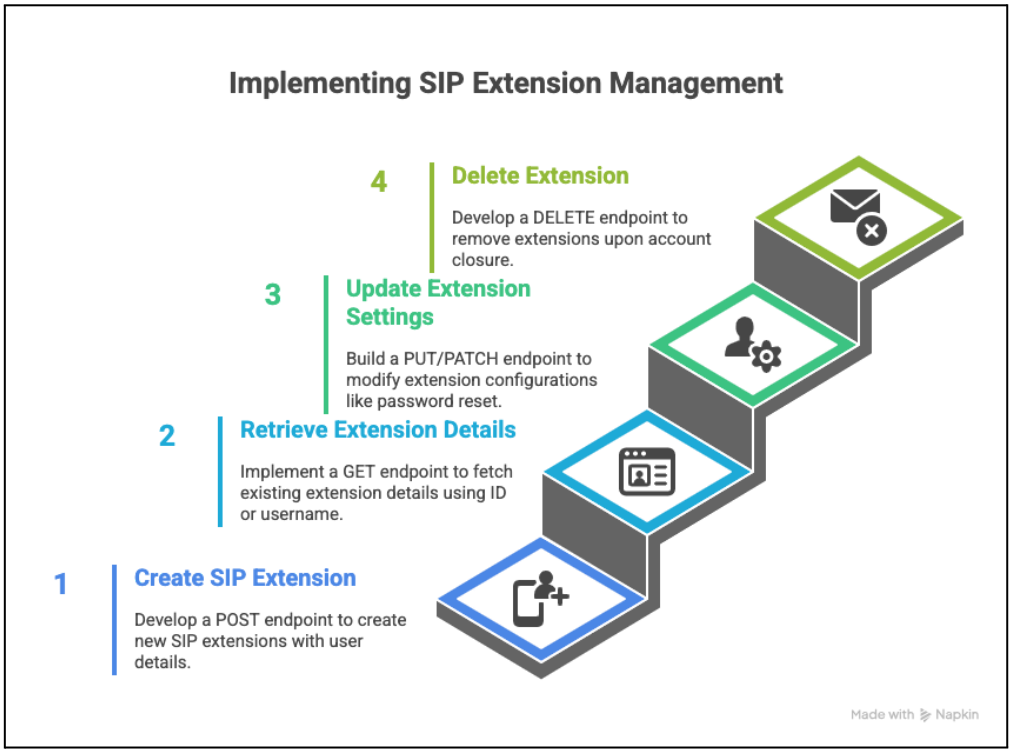

Fig. SIP Extension Management

Phone Number Management API

Our backend needs to let users search for and purchase phone numbers. We need SYSCONFiG's phone number API that allows us to search available numbers by filtering by country, area code, or city, see number type such as local, toll-free, or mobile, and view pricing for each number type. We need to purchase or provision numbers by associating them with a user's SIP extension and confirm the number is active and routing correctly. We need to configure number routing such as which SIP extension receives calls to this number and setting up failover or forwarding rules. We need to release or cancel numbers when users no longer want them, returning the number to the available pool.

Specific API Endpoints Required:

- GET endpoint for searching available numbers with query parameters for country code, area code, number type, and quantity
- POST endpoint to purchase a specific number with parameters linking it to a SIP extension ID
- GET endpoint to list all numbers owned by an account or extension
- PUT endpoint to update routing configuration for a number
- DELETE endpoint to release a number back to inventory

Beyond just tracking calls through webhooks, we also need direct API access to call detail records for historical data and backup purposes.

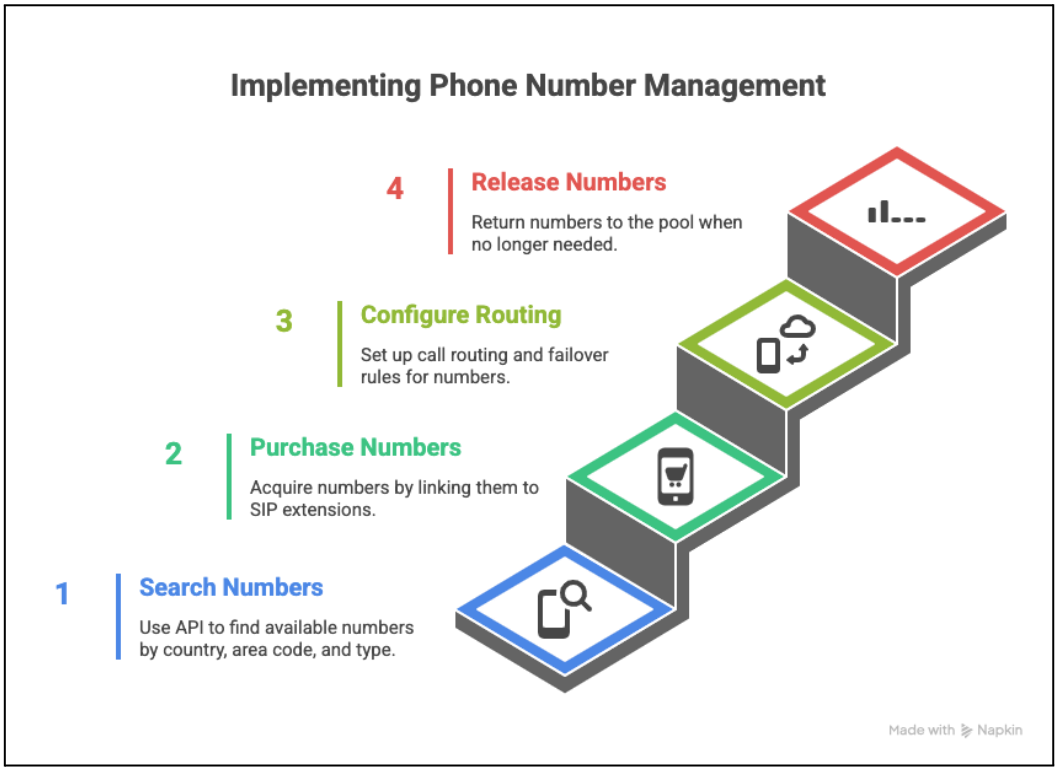

Fig. Phone Number Management

Call Detail Records API and Webhook

This is how we track every call for billing and history. We need two mechanisms from SYSCONFiG. First, webhook delivery where immediately after every call ends, SYSCONFiG posts call detail record data to our webhook endpoint URL. Second, we need the CDR data structure including unique call identifier, calling party number, called party number, call start timestamp, call end timestamp, duration in seconds, call cost in our agreed currency, call status such as completed, failed, busy, or no-answer, recording URL if call was recorded, and any additional metadata like codec used or disconnect reason.

Webhook Configuration Required:

- URL where SYSCONFiG will post CDR data to our server
- Authentication method such as shared secret, HMAC signature, or API key to verify webhooks genuinely come from SYSCONFiG
- Retry policy explaining what happens if our webhook endpoint is temporarily down
- Delivery guarantee such as at-least-once delivery meaning we might receive duplicate webhooks

With the backend APIs and integrations defined, we now turn to the technical infrastructure that actually makes voice calls possible across all networks and devices.

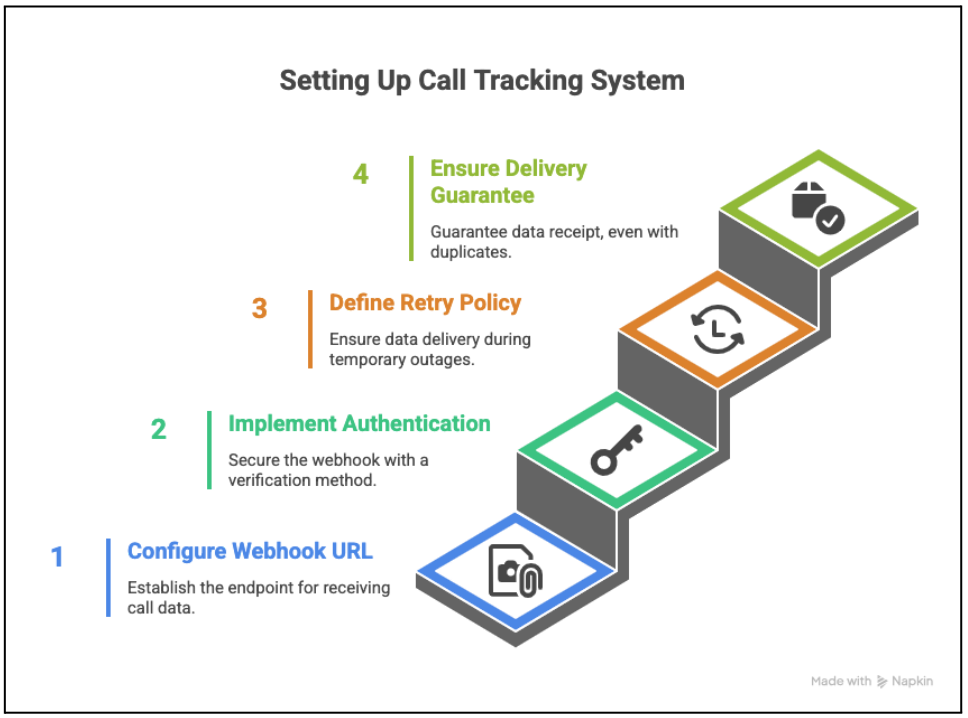

Fig. Setting Up Call Tracking System

STUN and TURN Server Details - Critical for Web Calling

WebRTC requires STUN and TURN servers for NAT traversal. Without these, calls fail on many networks. We need from SYSCONFiG their STUN server addresses and ports formatted as stun:stun.example.com:3478, TURN server addresses and ports as turn:turn.example.com:3478, TURN server credentials including username and password since TURN relaying requires authentication, supported protocols such as UDP, TCP, or TLS for TURN, and geographic locations of servers if they have multiple regions for latency optimization.

Why This Is Non-Negotiable: Approximately 20 to 30 percent of networks require TURN relay due to symmetric NAT or restrictive firewalls particularly in corporate environments. Without TURN configured, our web softphone would fail for those users. They'd try to make calls and nothing would happen. STUN helps with simpler NAT scenarios, but TURN is the

fallback that ensures calls always work. This is so critical that we need it before any call testing begins.

While our software applications use WebRTC for calling, some customers will prefer traditional desk phones. These physical devices need their own configuration approach.

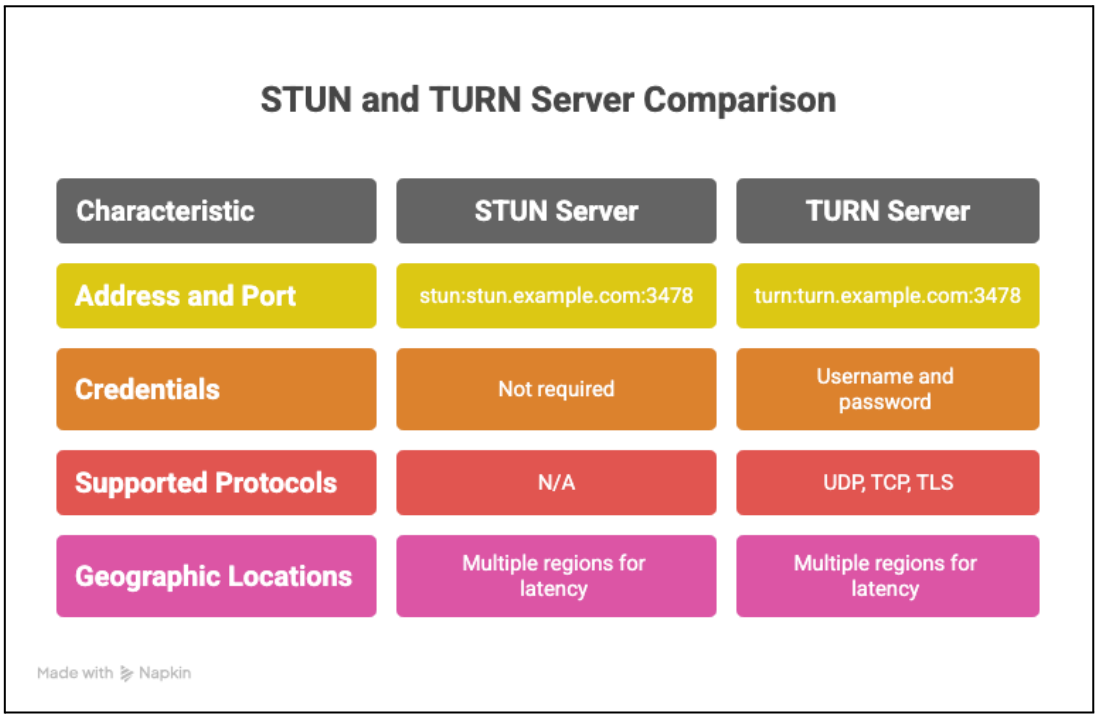

Fig. STUN and TURN Server Comparison

Device Auto-Provisioning Configuration

For physical IP phones, we need auto-provisioning support. This includes confirmation that SYSCONFiG's SIP infrastructure works with standard SIP phones from manufacturers like Yealink and Grandstream, provisioning server requirements if we need to run our own or if SYSCONFiG provides one, configuration file format typically XML or text-based for different phone models, and variables we need to include such as SIP username, password, server address, and extension settings.

Auto-Provisioning Flow: Phone powers on and looks for provisioning server via DHCP option 66 or hardcoded URL. Phone contacts provisioning server with its MAC address. Our server generates configuration file specific to that MAC address. Phone downloads config. Phone registers to SYSCONFiG using credentials from config. The phone is ready to use. This must happen automatically without user intervention.

Configuration Parameters:

- SIP account settings same as used by our apps

- Dial plan rules for number formatting and routing
- Function key programming for speed dial and features
- Codec preferences matching what SYSCONFiG supports
- NTP server and timezone settings
- Display language and regional settings

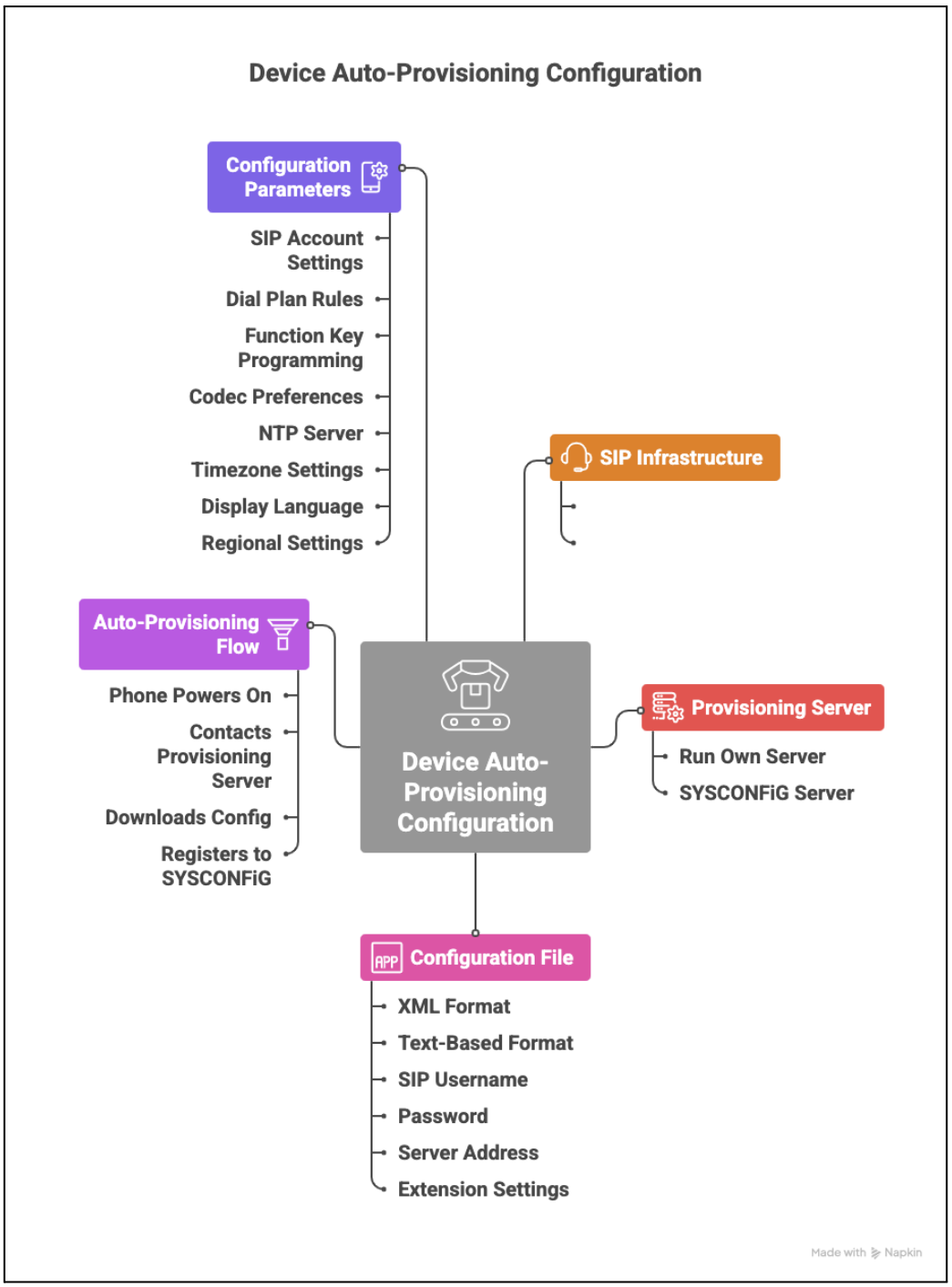

Fig. Device Auto Provisioning Configuration

Summary

The backend phase establishes all the foundational systems our platform needs to function. From user provisioning to phone number management, from real-time call tracking through webhooks to the technical infrastructure for reliable calling across any network, each component we've outlined is essential. Getting these integrations right with SYSCONFiG in the backend phase ensures that when we move to building user-facing applications in subsequent phases, we have a solid, tested foundation to build upon.